

MLIB Review & Set-Up

Nicholas Burgess



MHI

Nicholas.Burgess@uk.mizuho-sc.com

24th April 2017

Global Quant Analytics

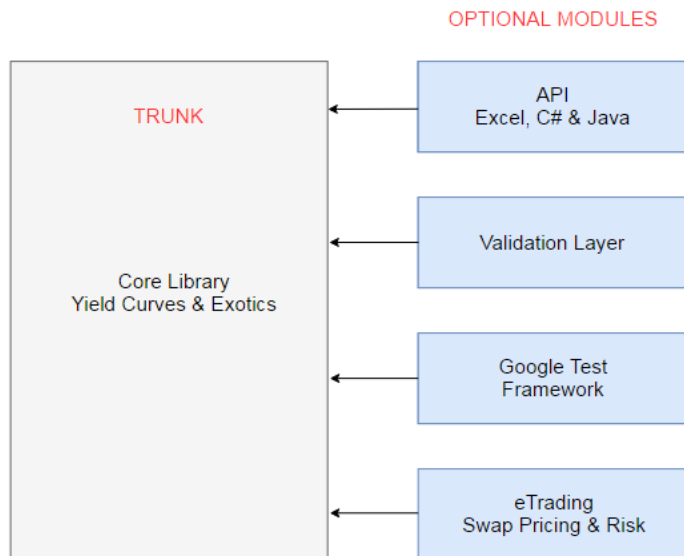
- MLIB is the Global Quant Analytics library at Mizuho
- Owned by MHSC Tokyo
- Co-Developed by MHI London & MCMC New York

Features

- Yield Curves with CSA Support
- Swaps Pricing & Risk: OIS, IRS, ASW, Basis, Xccy & Amortizers
- Bond Pricing: US Treasuries, German Bunds, Japanese JGBs ...
- IR Options: Swaptions, Caps, Floors ...
- Exotic Options: IR / FX Hybrids

- ① Library Structure
- ② Project Structure
- ③ Folder Structure
- ④ Environment Variables
- ⑤ Automation Scripts
- ⑥ External Libraries
- ⑦ MSBuild Property Files
- ⑧ Set-Up Instructions
- ⑨ Utilities: MLIB Dojo, Test Ninja & GodzillaVsMLIB
- ⑩ Appendix

Library Structure



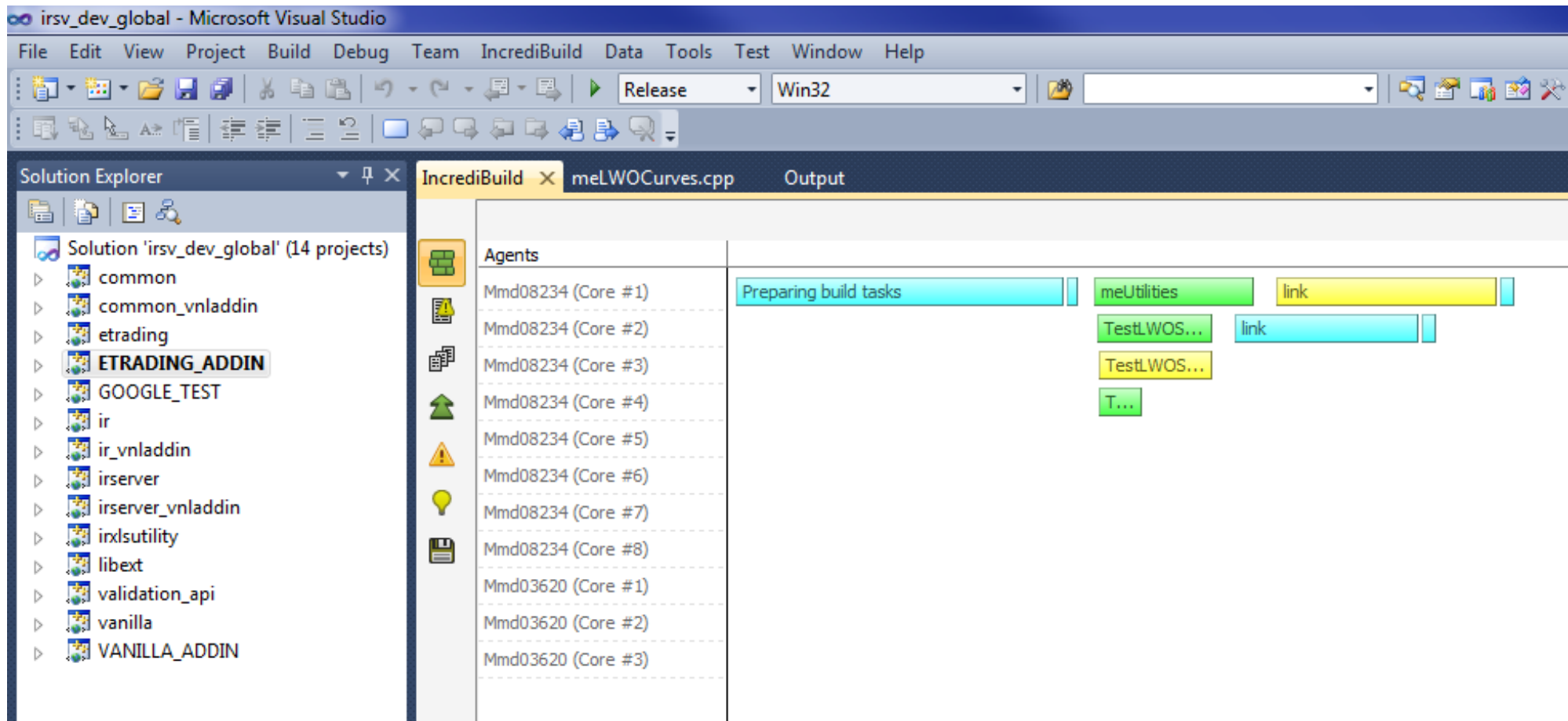
Project Structure

- Core: Yield Curve & Exotics
- Optional Projects: Modular & Independent

Solution Files

- **irsv_dev.sln:**
Original native solution for Tokyo use, excludes optional modules
- **irsv_dev_global.sln:**
For global use and includes optional modules
- **irsv_dev_everything.sln:**
Includes everything, both optional modules & Tokyo server projects

irsv_dev_global.sln

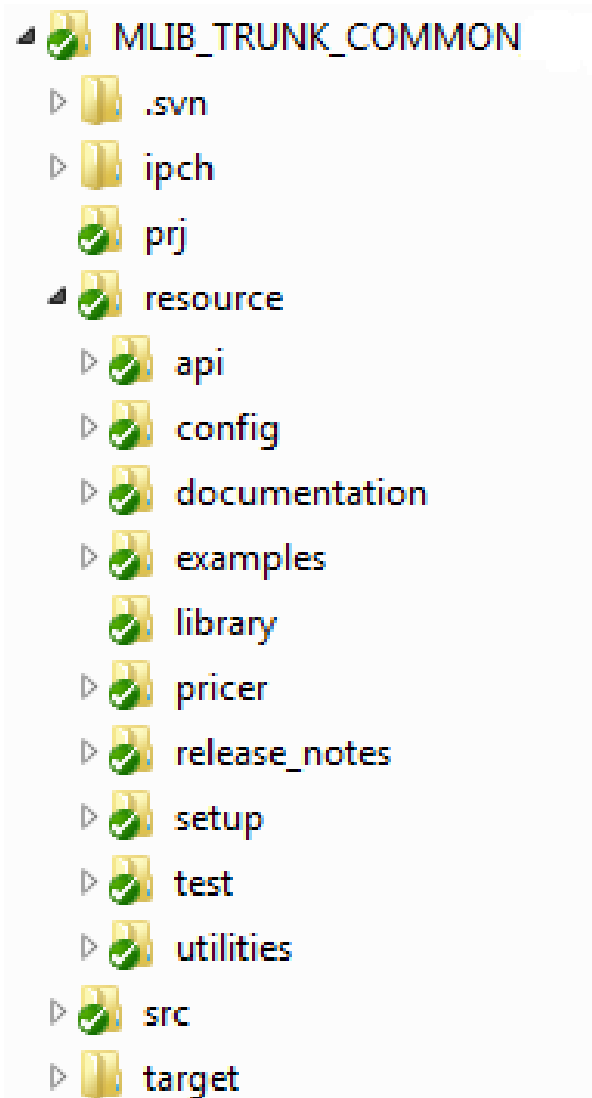


Library rebuilds in 3 minutes using Incredibuild

- **common**: Infrastructure & Common Utilities: Entity Pool Object Store, Date, Interpolation, Quadrature, Brownian Bridge et al
- **common_vnladdin**: *.lib creation for common project
- **ir**: IR Models
- **ir_vnladdin**: *.lib creation for ir project
- **irserver**: Server code for Model Calibration & Risk
- **irserver_vnladdin**: *.lib creation for irserver project
- **irxlutility**: Excel utility functions for ir project
- **libext**: External Library Optimization / Calibration
- **vanilla**: Library Set-up for Vanilla Addin
- **VANILLA_ADDIN**: Vanilla Add-In Excel API

- **etrading:**
LW Objects, Swap and Bond Pricing & Risk
- **ETRADING_ADDIN:**
Etrading Add-In Excel API
- **GOOGLE_TEST:**
Google Test Harness
- **validation_api:**
API validation layer; Single Entry Point for all API Function Calls

Folder Structure



Main Folders

prj: Project Files

src: Source Code

target: Object Files & Binaries

resource Folder

- **api**: C# and Java API source code
- **config**: calendars and property config files
- **documentation**: docs go here
- **examples**: excel example templates
- **library**: optional external library folder
- **pricer**: excel pricing workbooks
- **set-up**: set-up notes and *.props files
- **test**: google test case folder
- **utilities**: batch files, Excel toolbar etc

Environment Variables

Environment variables have been introduced to allow MLIB to be used and configured for global use.

MLIB

This environment variable sets where the library is located. Project configuration and property files are referenced relative to this root folder.

MLIB_EXTERNAL_LIB_PATH

This sets the location reference for the external libraries such as Google Test, Boost and SwigWin and allows us to have multiple check-outs of the source code, whilst only having to check-out the external libraries once.

MLIB_LOCATION

This variable is used by the automation scripts to load and configure regional folder and other settings.

Environment Variable Setting

Environment variables can be set automatically by clicking on the following batch file scripts. Administrator access is **not** required.

Batch File Folder

`$MLIB$\resource\utilities\batchfiles\`

Setting Environment Variables

`$MLIB$\resource\utilities\batchfiles\SetMlibEnvironmentVariables\`

`Set_MLIB_EnvironmentVariable.bat`

`Set_MLIB_EXTERNAL_LIB_PATH.bat`

`Set_MLIB_LOCATION.bat`

Batch File Folder

`$MLIB$\resource\utilities\batchfiles\`

Automation Scripts

- Build MLIB
- Build + Test
- Deploy Add-In
- Generate Build Logs
- Generate Release Notes
- Run Google Tests
- SVN Update
- Update + Build + Run Tests

External Libraries

- Boost 1.55.0
- 'Eigen' Matrix Analytics
- Google Mock 1.7.0
- Google Test 1.7.0
- Google Test Guitar (C# Test App)
- Google Protobuf 2.6.1
- QuantLib 1.3
- Rapid JSON
- SwigWin 3.0.8
- ZeroMQ 4.0.4

SwigWin API Generator

The SwigWin toolkit auto-generates sourcecode and binaries for a wide variety of APIs including C#, Java, R and Python.

Requirements

- Install SwigWin.exe
- Ensure SwigWin 3.0.8 is in your third party folder and includes the swig.exe
- Install the Java **JDK-8u101-windows-i586** package
- Add the appropriate Java Build for JNI
- See [\\$MLIB\\$\src\MLIB_CLIENT_API\source\README.txt](#) for full installation details and instructions

Property Files, *.props

MSBuild property files *.props are used to control legacy project settings. Props files should be copied and pasted as listed below. We would like to deprecate this procedure in the future and explicitly set all project properties directly within MLIB projects.

Copy Location

`$MLIB$\resource\setup\vs_properties\Microsoft.Cpp.Win32.user.props`

`$MLIB$\resource\setup\vs_properties\Microsoft.Cpp.x64.user.props`

Paste Location

`C:\Users\Username\AppData\Local\Microsoft\MSBuild\v4.0`

Set-Up Instructions

A summary of the set-up process is outlined below. For full instructions see [\\$MLIB\\$\resource\set-up\README.txt](#)

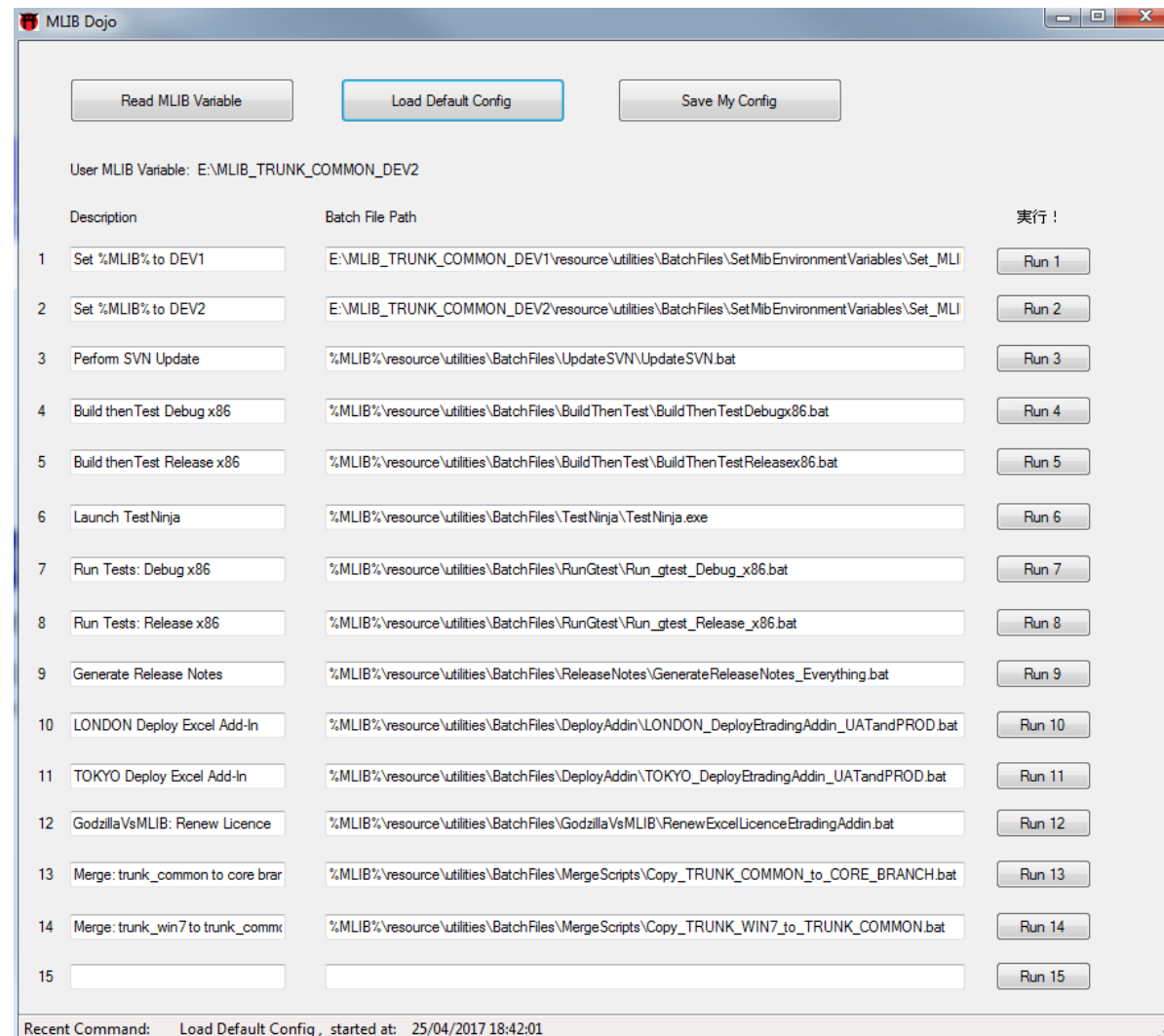
- ① Check-Out the Sourcecode¹
- ② Check-Out the Third-Party External Libraries²
- ③ Set the 3 environment variables
(see slides [MLIB Configuration & Automation Scripts I](#))
- ④ Update the MSBuild Prop Files (see slides [MLIB Prop Files](#))
- ⑤ Install SwigWin and the Java SDK (see slides [SwigWin API](#))
- ⑥ Install XLL+, Visual Studio Plugin for Excel Add-In Generation
- ⑦ Install Incredibuild, reduces rebuild time from 3 hours to 3 mins

¹http://10.153.2.89/repos/M-Library/branches/irsv_dev/trunk_common

²http://10.153.2.89/repos/M-Library/branches/irsv_dev/library

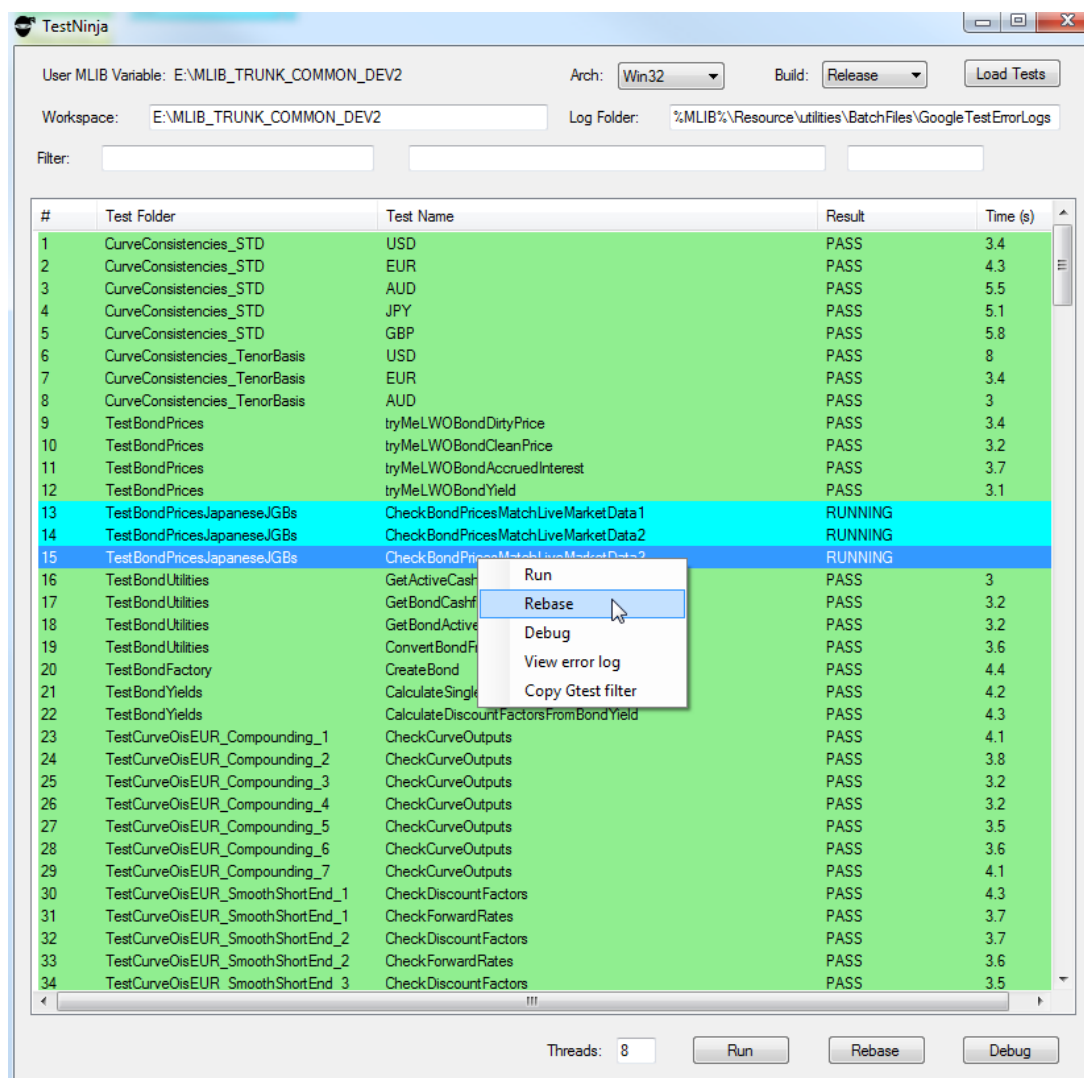
Automation Script Dashboard

The MLIB Dojo is used to centrally manage and trigger automation scripts.



Parallelized Unit- & Regression Test App

The Test Ninja app is used to run, reset and debug unit & regression tests.



MLIB Excel Add-In License Regenerator

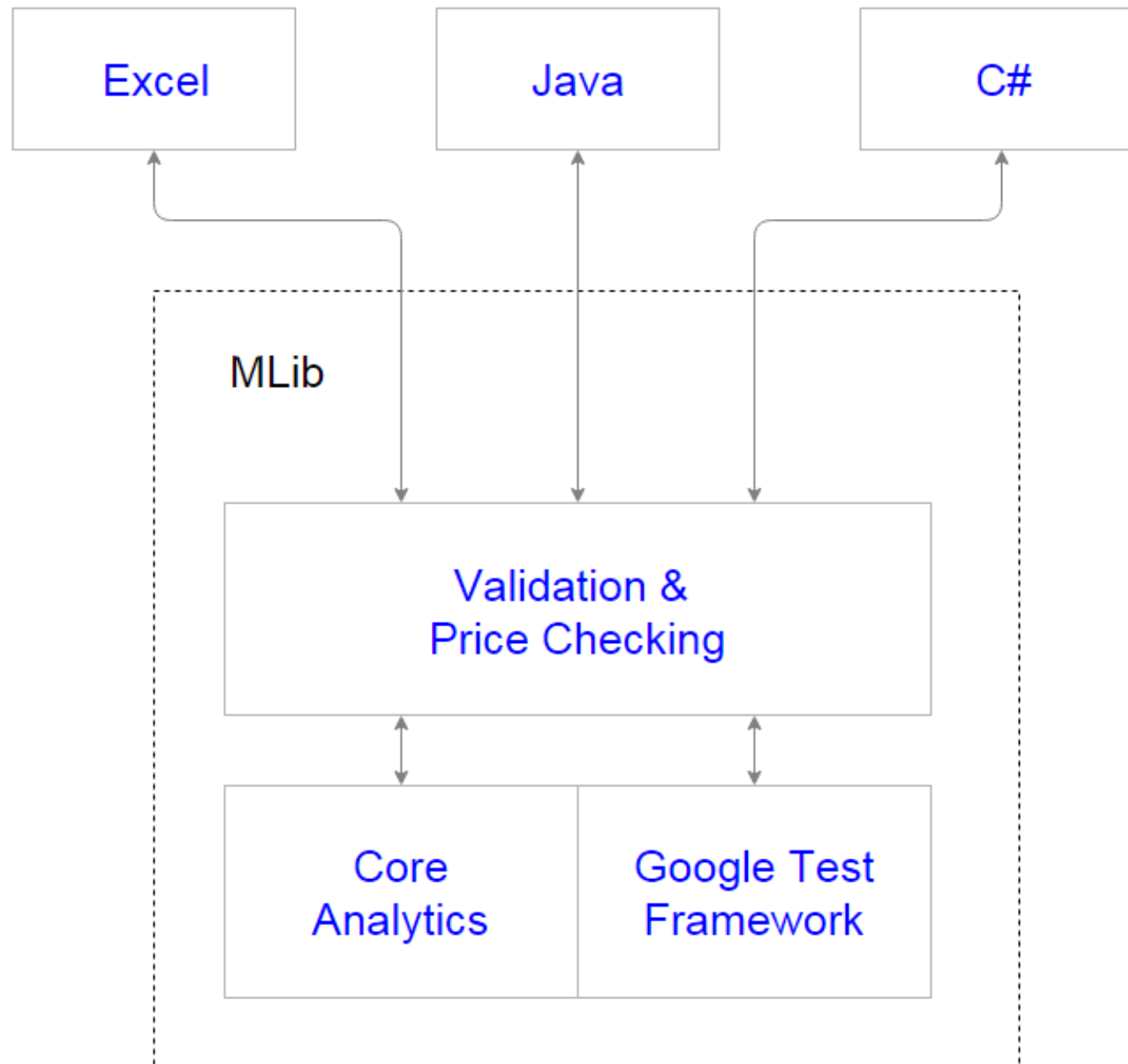
MLIB Excel Add-Ins expire after 3 months. The GodzillaVsMLIB executable refreshes the license of expired Excel Add-Ins.



- Simply drag an existing MLIB Excel Add-In on top of the executable
- A new add-in will be generated with a renewed license
- Additionally a back-up of the original Add-In will be created

Appendix

Appendix 1: Project Flow Diagram



Purpose: Simplicity, Transparency, Configurability, Serialization

Features

① Date Generation

Generation of schedules, year fractions and dates

② Extended Curve Building with CSA Support

③ Simplified Discount Factor & Forward Calculations

④ Swap Pricing

OIS, IRS, Tenor Basis, Xccy, Asset Swaps, Amortizers etc.

⑤ Detailed Cashflow, Price & Risk Breakdowns

⑥ Object & Serialization Support

Data Grids, Fixing Tables, Fee Schedules, Swaps & Curve Serialization

⑦ Test Framework, Logging, Record and Replay Features

⑧ Build Automation, Test and Deployment Scripts

Test Fixtures & Google Test Requirements

To compliment Google Test "[test fixtures](#)" or classes are used for large complex model tests. Models and yield curves can be recorded, loaded, preloaded, updated and re-based. This project works in conjunction with the "[validation_api](#)" project.

Features

- [Approx 300 Test Cases Comprising a Total of 5,000 Tests](#)
- [Test Fixture Classes for More Complex Tests](#)
- [Test Recording & Playback](#)
- [Automated Test Scripts](#)

Overview of Validation_API Project

The validation project is a thin project layer that sits on top of the analytics. The validation layer provides a single common entry point to the analytics. This ensures that all client applications and APIs such as Excel, C#, Java consistently return the same results. We consistently perform all input validation and application crash checks in this project.

Features

- Validation of User Inputs
- Single Entry Point into Analytics for all Client Apps
- Test Recording & Playback
- Excel and other API Application Crash Checks & Controls

Overview

This project is an interface generator used to generate binaries to allow external libraries written in other languages to connect to MLIB.

Features

- Accessible via **irsv_dev_everything.sln**
- [Swigwin](#)
The project uses the Swigwin open-source package
- [C# & Java](#)
Currently this project generates API code to support C# and Java
- [26 Languages](#)
The framework supports up to 26 different coding languages
- [Future Work](#)
R and Python API support is likely to be added soon

The End